

AI Alapú Bőr- és Szín Elemzés Platform

Full-stack webalkalmazás Machine Learning integrációval

Tartalomjegyzék

- 1. [Projekt Áttekintése](#)
- 2. [Technológia Stack](#)
- 3. [Architektúra](#)
- 4. [Adatbázis](#)
- 5. [Backend API](#)
- 6. [Frontend Architektúra](#)
- 7. [Biztonsági Megoldások](#)
- 8. [Titkosítás és Hitelesítés](#)
- 9. [Machine Learning Modellek](#)
- 10. [Felhasználói Folyamatok](#)
- 11. [Telepítés és Futtatás](#)

Projekt Áttekintése

AI-alapú webalkalmazás, amely a felhasználók bőrtípusát és bőrproblémáit elemzi mesterséges intelligencia segítségével, valamint meghatározza az ideális szín-évszakot (color season). Az alkalmazás:

- **Bőrelemzés:** Deep Learning modell segítségével azonosítja a bőrtípust (zsíros, száraz, normál, kombinált)
- **Bőrprobléma-detektálás:** Felismeri a bőrhibákat (pattanások, karikák, stb.)
- **Szín-évszak analízis:** AI chatbot segítségével határozza meg a felhasználó szín-évszakát
- **Személyre szabott tanácsok:** Ajánlásokat ad a skincare protokollokról és kezelési módokról

Fő Funkciók

Felhasználó regisztráció és bejelentkezés (JWT auth)
AI-alapú bőrelemzés képfeltöltésből
Interaktív szín-évszak meghatározás chatbot-tal
Elemzési eredmények tárolása és visszahozása
Skincare protokollok és ajánlások
Felhasználói profil kezelés

Technológia Stack

Backend

Komponens	Technológia	Verzió	Célja
-----------	-------------	--------	-------

Komponens	Technológia	Verzió	Célja
Runtime	Node.js	LTS	JavaScript backend futtatás
Framework	Express.js	4.x	HTTP szerver és routing
Adatbázis	MySQL	8.x	Relációs adatok tárolása
ORM	Sequelize	6.x	Adatbázis absztrakció
Auth	JWT (jsonwebtoken)	9.x	Token alapú hitelesítés
Hashing	bcrypt	5.x	Jelszó titkosítás
ML API	Google Gemini	v1	AI alapú elemzés
Email	Nodemailer	6.x	Email küldés
Environment	dotenv	16.x	Konfigurációs változók

Frontend

Komponens	Technológia	Verzió	Célja
Framework	Vue.js	3.x	UI framework (Composition API)
Build Tool	Vite	5.x	Fejlesztési szerver és bundling
HTTP Client	Axios	1.x	API kommunikáció
Router	Vue Router	4.x	Client-side routing
Styling	SCSS	-	CSS preprocessor
Validator	Szokott validációs logika	-	Frontend form validáció

Machine Learning

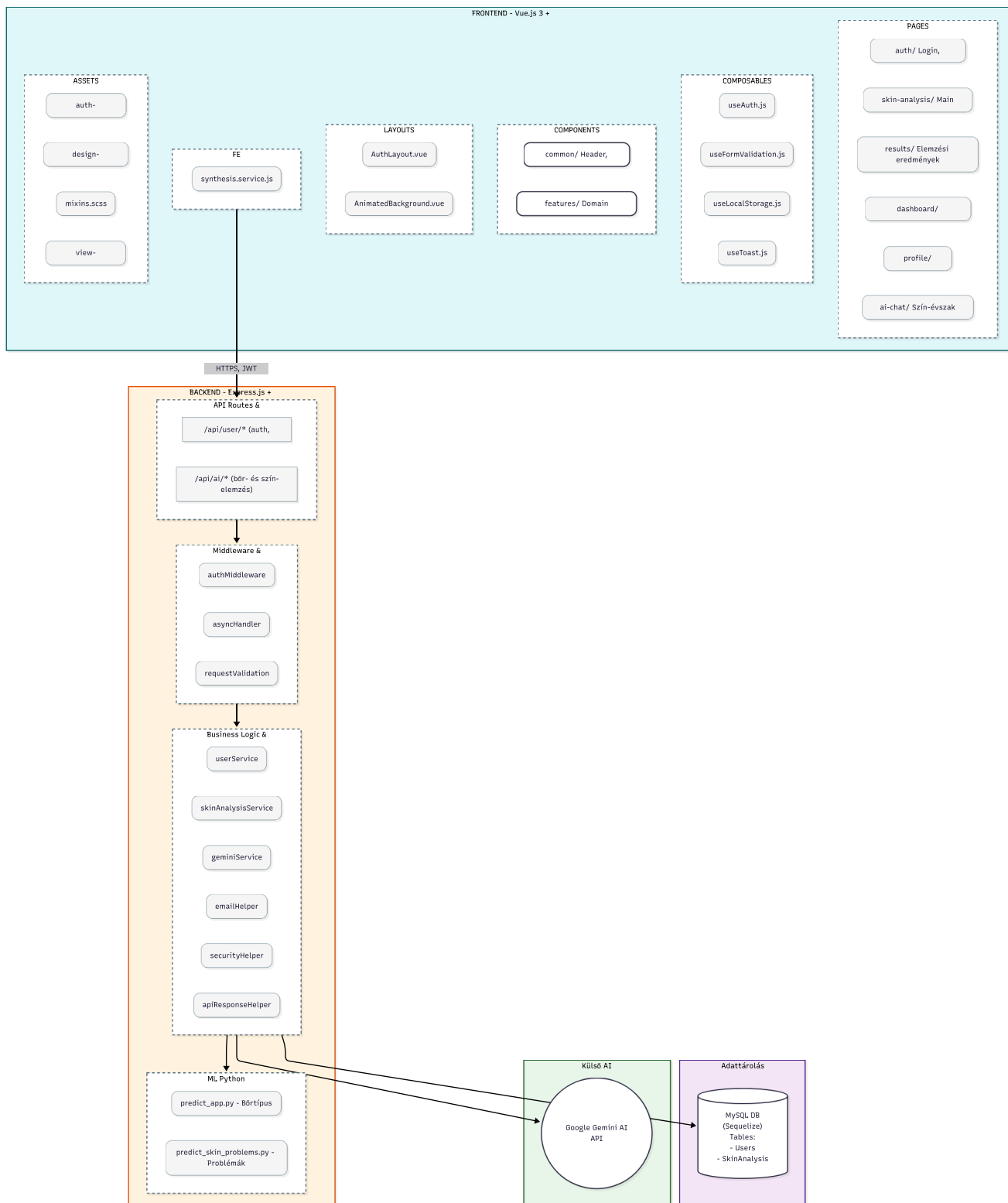
Komponens	Technológia	Verzió	Célja
Framework	TensorFlow/Keras	2.x	Deep Learning modellek
Nyelv	Python	3.9+	ML kód fejlesztés
Adatok	NumPy, Pandas	-	Adatfeldolgozás
Vizualizáció	Matplotlib	-	Modell tesztelés

Fejlesztési Eszközök

- **Version Control:** Git
- **Linting:** ESLint (Frontend)
- **Package Manager:** npm
- **Database Client:** MySQL Workbench

Architektúra

Rendszer Felépítése



MVC Minta

Routes (Express)



Controllers (user.controller.js, ai.controller.js)

↓
Services (user.service.js, skinAnalysisService)
↓
Models (Sequelize ORM - User, SkinAnalysis, ColorSeason)
↓
Database (MySQL)

Adatbázis

Adatbázis Séma

account (User) Tábla

```
CREATE TABLE account (  
  account_id INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  login_name VARCHAR(50) UNIQUE NOT NULL,  
  login_password_hash VARCHAR(200) NOT NULL,          -- hash  
  email_address VARCHAR(200) NOT NULL UNIQUE,  
  full_name VARCHAR(200) NOT NULL,  
  password_recovery_hash VARCHAR(200),                -- hash  
  password_recovery_expires DATETIME,                  -- Token lejárat  
  color_season_id INT,  
  color_analysis_date DATETIME,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

Titkosítva:

- login_password_hash - bcrypt (salt rounds: 10)
- password_recovery_hash - bcrypt (salt rounds: 10)

Nyilvántartott adatok:

- Felhasználónév, email, teljes név
- Szín-évszak azonosító
- Jelszó visszaállítás számára szükséges tokenek

skin_analysis Tábla

```
CREATE TABLE skin_analysis (  
  analysis_id INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  account_id INT UNSIGNED NOT NULL,                    -- FK -> account  
  skin_type ENUM('oily', 'dry', 'normal', 'combination') NOT NULL,  
  skin_problems JSON,                                  -- Probléma tömb  
  analysis_date DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
FOREIGN KEY (account_id) REFERENCES account(account_id) ON DELETE CASCADE  
);
```

Titkosítva: Nem

JSON Szerkezet:

```
["Acne", "Bags", "Redness"]
```

color_seasons Tábla

```
CREATE TABLE color_seasons (  
  season_id INT PRIMARY KEY AUTO_INCREMENT,  
  season_name VARCHAR(20) UNIQUE NOT NULL,           -- 'spring', 'summer'  
  season_display_name VARCHAR(50) NOT NULL,         -- 'Tavaszi', 'Nyár'  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

Titkosítva: Nem

Előre definiált adatok:

- spring (Tavaszi)
- summer (Nyár)
- autumn (Ősz)
- winter (Tél)

Relációk

```
User (1) → (N) SkinAnalysis  
User (1) → (1) ColorSeason
```

Backend API

API Végpontok

User routes

1. Bejelentkezés

```
POST /api/user/login
Content-Type: application/json
```

```
{
  "username": "john_doe",
  "password": "SecurePass123"
}
```

Válasz (200 OK):

```
{
  "success": true,
  "message": "Login successful",
  "user": { "id": 1, "username": "john_doe", "email": "john@example.com" },
  "token": "eyJhbGc..."
}
```

Válasz (401 Unauthorized):

```
{
  "success": false,
  "message": "Bejelentkezési hiba",
  "errorCode": "INVALID_CREDENTIALS"
}
```

Mit csinál:

1. Felhasználónév és jelszó alapján keresi a user-t
2. Jelszó ellenőrzés bcrypt-tal
3. JWT token generálása 24 órás lejárattal
4. User adatok és token visszaadása

Titkosítás: JWT (SHA256)

2. Regisztráció

```
POST /api/user/register
Content-Type: application/json
```

```
{
  "username": "john_doe",
  "password": "SecurePass123",
  "email": "john@example.com",
  "fullName": "John Doe"
}
```

Válasz (200 OK):

```
{
  "success": true,
  "message": "Registration successful",

```

```
"user": { "id": 1, "username": "john_doe", ... }
}
```

Válasz (409 Conflict):

```
{
  "success": false,
  "message": "",
  "errorCode": "USERNAME_TAKEN" | "EMAIL_TAKEN"
}
```

Mit csinál:

1. Input validáció
2. Felhasználónév és email egyediségének ellenőrzése
3. Jelszó bcrypt-tal való hashelése
4. User létrehozása az adatbázisban

Titkosítás: Jelszó - bcrypt

3. Elfelejtett jelszó

```
POST /api/user/forgot-password
{
  "email": "john@example.com",
  "emailTemplate": "recovery-html"
}
```

Válasz (200 OK):

```
{ "success": true, "message": "Password recovery email sent" }
```

Mit csinál:

1. Email alapján megkeresi a user-t
2. Generál egy 32 bájtos random token
3. Token hashelése bcrypt-tal
4. Tárolás az adatbázisban 24 órás lejárattal
5. Reset link elküldése emailben

Titkosítás: Token - bcrypt hash

4. Jelszó Visszaállítás

```
POST /api/user/reset-password
{
  "token": "abc123xyz...",
}
```

```
"password": "NewSecurePass123"
}

Válasz (200 OK):
{ "success": true, "message": "Password reset successful" }

Válasz (400 Bad Request):
{
  "success": false,
  "message": "",
  "errorCode": "INVALID_TOKEN" | "TOKEN_EXPIRED"
}
```

Mit csinál:

1. Token ellenőrzése az adatbázisban
2. Lejárat ellenőrzése
3. Új jelszó bcrypt-tal való hashelése
4. Token nullázása

Titkosítás: Jelszó - bcrypt

5. Profil Lekérése

```
GET /api/user/me
Authorization: Bearer <JWT_TOKEN>

Válasz (200 OK):
{
  "success": true,
  "message": "Profile retrieved",
  "user": { ... }
}
```

Mit csinál:

1. JWT token ellenőrzése
 2. User adatok lekérése az adatbázisból
-

6. Profil Frissítés

```
PUT /api/user/profile
Authorization: Bearer <JWT_TOKEN>
{
  "fullName": "John Updated",
```



```
"username": "john_new",  
"email": "john.new@example.com"  
}
```

Mit csinál:

1. JWT token ellenőrzése
 2. Adatok frissítése az adatbázisban
-

7. Jelszó Változtatás

```
PUT /api/user/change-password  
Authorization: Bearer <JWT_TOKEN>  
{  
  "currentPassword": "OldPass123",  
  "newPassword": "NewPass456"  
}
```

Mit csinál:

1. Jelenlegi jelszó ellenőrzése
 2. Új jelszó bcrypt-tal való hashelése
-

8. Fiók Törlése

```
DELETE /api/user/delete-account  
Authorization: Bearer <JWT_TOKEN>
```

Mit csinál:

1. Fiók és összes kapcsolódó adat törlése
 2. CASCADE delete a skin_analysis táblából
-

9. Szín-Évszak Frissítés

```
PUT /api/user/color-season  
Authorization: Bearer <JWT_TOKEN>  
{  
  "season": "spring"  
}
```

Válasz (200 OK):

```
{
  "success": true,
  "message": "Color season updated",
  "user": { "colorSeason": { "id": 1, "name": "spring", "displayName": "Tavaszi" } }
}
```

Mit csinál:

1. Az adott évszak ID lekérése
2. User color_season_id frissítése

10. Elemzési Eredmények Lekérése

```
GET /api/user/analyses-results
Authorization: Bearer <JWT_TOKEN>

Válasz (200 OK):
{
  "success": true,
  "message": "Analyses results retrieved",
  "colorSeason": { ... },
  "skinAnalysis": {
    "id": 1,
    "skinType": "oily",
    "skinProblems": ["Acne", "Redness"],
    "analyzedAt": "2025-10-15T10:30:00Z"
  },
  "protocol": { ... },
  "problemsProtocol": { ... }
}
```

Mit csinál:

1. JWT token ellenőrzése
2. Legutóbbi bőrelemzés lekérése
3. Szín-évszak adatok lekérése
4. Skincare protokollok betöltése

Ai elemzés

1. Bőrelemzés (Képfeltöltés)

```
POST /api/ai/analyze-skin
Authorization: Bearer <JWT_TOKEN>
```

```
Content-Type: multipart/form-data
File: <image.jpg>
```

Válasz (200 OK):

```
{
  "success": true,
  "message": "Skin analysis completed",
  "analysis": {
    "skinType": "oily",
    "skinProblems": {
      "detected": [
        { "problem": "Acne", "confidence": 0.95, "name_hu": "Pattanások" },
        { "problem": "Redness", "confidence": 0.87, "name_hu": "Vörösség" }
      ]
    }
  },
  "protocol": { ... },
  "problemsProtocol": { ... }
}
```

Mit csinál:

1. JWT token ellenőrzése
2. Kép feldolgozása
3. Bőrtípus meghatározása (TensorFlow modell)
4. Bőrproblémák detektálása (TensorFlow modell)
5. Skincare protokollok betöltése
6. Adatbázisban tárolás vagy frissítés (felhasználónként egy rekord)

Python ML Modellek:

- `skin_accurate_model.keras` - Bőrtípus predikció
- `skin_problems_model.keras` - Bőrprobléma detektálás

Titkosítva: Nem (képek ideiglenes fájlok, nem tárolódnak)

2. AI Chat (Szín-Évszak Elemzés)

```
POST /api/ai/chat
Authorization: Bearer <JWT_TOKEN>
{
  "message": "Valami",
  "conversationHistory": [ ... ],
  "context": null
}
```

Válasz (200 OK):

```
{
  "success": true,
```

```
"message": "Chat response received",
"response": "A leírásod alapján...",
"updatedHistory": [ ... ]
}
```

Mit csinál:

1. JWT token ellenőrzése
2. Üzenet Gemini API-nak küldése
3. Conversation history fenntartása
4. Válasz streamelése

AI Provider: Google Gemini API

3. Szín-Évszak Meghatározás (Analyze Color)

```
POST /api/ai/analyze-color
Authorization: Bearer <JWT_TOKEN>
{
  "conversationHistory": [ ... ],
  "accountId": 1
}

Válasz (200 OK):
{
  "success": true,
  "message": "Color analysis completed",
  "season": "spring",
  "confidence": 0.92
}
```

Mit csinál:

1. Conversation history elemzése
2. AI által javasolt szín-évszak meghatározása
3. Felhasználó adatok frissítése

Middleware-ek és Validáció

authenticateToken Middleware

```
// Backend/service/middlewares/auth.middleware.js
- JWT token kinyerése az Authorization headerből
- Token verifikálása JWT_SECRET-tel
```

- Felhasználó adatok (id, username, email) req.user-be kerülésre
- 401 vagy 403 hiba logika

validateRequired Middleware

```
// Backend/service/middlewares/request-validation.middleware.js
```

- Input mezők kötelezőségének ellenőrzése
- Type checking

asyncHandler Middleware

```
// Backend/service/middlewares/async-handler.middleware.js
```

- Promise/async/await hibakezelés
- Try-catch automata burkolás

Frontend Architektúra

Projekt Szerkezete

```
Frontend/
├── src/
│   ├── App.vue
│   ├── main.js
│   ├── assets/
│   │   ├── auth-common.css
│   │   ├── design-system.css
│   │   ├── mixins.scss
│   │   └── view-common.css
│   ├── components/
│   │   ├── common/
│   │   │   ├── layout/
│   │   │   ├── forms/
│   │   │   ├── feedback/
│   │   │   └── base/
│   │   └── features/
│   ├── composables/
│   │   ├── useAuth.js
│   │   ├── useFormValidation.js
│   │   ├── useLocalStorage.js
│   │   └── useToast.js
│   ├── data/
│   │   ├── dashboard-content.js
│   │   └── validation-messages.js
│   └── layouts/
```

```

├── AnimatedBackground.vue
├── AuthLayout.vue
├── pages/
│   ├── auth/
│   │   ├── LoginPage.vue
│   │   ├── RegisterPage.vue
│   │   ├── ForgotPasswordPage.vue
│   │   └── ResetPasswordPage.vue
│   ├── dashboard/
│   ├── skin-analysis/
│   │   └── SkinAnalysisPage.vue
│   ├── results/
│   │   └── ResultsPage.vue
│   └── profile/
├── router/
│   └── index.js
├── services/
│   └── index.js
├── package.json
├── vite.config.js
└── vercel.json

```

Composables (Composition API Hooks)

useAuth.js

Felelőssége: Felhasználó hitelesítés

Funkciók:

```

const {
  login,           // Bejelentkezés
  logout,          // Kijelentkezés
  register,        // Regisztráció
  forgotPassword,  // Jelszó elfelejtés
  resetPassword,   // Jelszó visszaállítás
  errorMessage,    // Hibaüzenet ref
  successMessage,  // Sikerüzenet ref
  checkQueryMessages, // Query params olvassa
  setError,        // Error setter
  setSuccess       // Success setter
} = useAuth()

```

Tárolás:

- **authToken** - localStorage-ban
- **authUser** - localStorage-ban és memóriában
- JWT token kiterjesztése: 24 óra

useLocalStorage.js

Felelőssége: LocalStorage adatok kezelése

Funkciók:

```
const {
  getUserData,      // User adatok lekérése
  getUserId,        // User ID lekérése
  getAnalysisResult, // Elemzési eredmények cache
  setAnalysisResult // Elemzési eredmények mentése
} = useLocalStorage()
```

Tárolt kulcsok:

- `authToken` - JWT token
 - `authUser` - User objektum
 - `aiLastAnalysisResult_{userId}` - Bőrelemzés cache
-

useFormValidation.js

Felelőssége: Form validáció

Funkciók:

- Email validáció (regex)
 - Jelszó validáció (min 8 karakter, nagybetű, kisbetű, szám)
 - Felhasználónév validáció (3-20 karakter, csak alfanumerikus)
 - Custom validator támogatás
-

useToast.js

Felelőssége: Felhasználói értesítések

Funkciók:

- Sikerüzenetek megjelenítése
 - Hibaüzenetek megjelenítése
 - Auto dismiss időzítő
-

Services (API kommunikáció)

apiClient (Axios)

```
// Frontend/src/services/index.js

// Interceptor: JWT token automatikus hozzáadása
apiClient.interceptors.request.use(config => {
  const token = localStorage.getItem('authToken')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

// Error handling
apiCall(axiosCall, errorMessage) → { success, data, message }
```

userService

```
// Felhasználó-specifikus API hívások
await userService.login(username, password)
await userService.register(userData)
await userService.getProfile()
await userService.changePassword(current, new)
await userService.forgotPassword(email)
await userService.resetPassword(token, password)
await userService.getAnalysesResults()
```

aiService

```
// AI-specifikus API hívások
await aiService.chat(message, conversationHistory, context)
await aiService.chatWithImage(imageBase64, prompt, history, context)
await aiService.analyzeColorType(conversationHistory, accountId)
```

Oldalak

SkinAnalysisPage.vue

Cél: Bőrelemzés (képfeltöltés és eredmények)

Állapotok:

1. **isLoadingPage = true** → Adatok betöltése
 - LocalStorage ellenőrzése
 - Backend API lekérés (GET /api/user/analyses-results)
2. **analysisResult = null** → Feltöltő UI
 - Kép kiválasztása

- Preview megjelenítése
- 3. **isAnalyzing = true** → Spinner
- 4. **analysisResult ≠ null** → Eredmények UI
 - Bőrtípus megjelenítése
 - Problémák felsorolása
 - Protokoll betöltése
 - Cache mentése

Validáció:

- Fájl méret: max 10MB
 - Formátum: JPG, PNG csak
 - kötelező: kép feltöltése
-

LoginPage.vue

Cél: Felhasználó bejelentkezés

Validáció:

- Felhasználónév: kötelező
- Jelszó: kötelező
- Backend hibák lefordítása (**translateError**)

Flow:

1. Form kitöltés
 2. Frontend validáció
 3. POST /api/user/login
 4. Token és user adatok localStorage-ba
 5. Dashboard-ra navigálás
-

RegisterPage.vue

Cél: Új felhasználó regisztráció

Validáció:

- Felhasználónév: 3-20 karakter, alfanumerikus
 - Jelszó: min 8 karakter, nagybetű, kisbetű, szám
 - Email: valid email formátum
 - Teljes név: kötelező
 - Jelszó megerősítés: egyeznie kell
-

ResultsPage.vue

Cél: Elemzési eredmények megjelenítése

Tartalom:

- Szín-évszak információk
 - Skincare protokoll ajánlások
 - Bőrprobléma-specifikus tanácsok
-

Biztonsági Megoldások

1. Hitelesítés (Authentication)

JWT (JSON Web Token)

- **Hol:** `Backend/service/helpers/security.helper.js`
- **Algoritmus:** HMAC SHA256
- **Secret:** `process.env.JWT_SECRET` (fallback: `'fallback-secret-key-change-in-production'`)
- **Lejárat:** 24 óra
- **Payload:**

```
{
  "id": 1,
  "username": "john_doe",
  "email": "john@example.com",
  "iat": 1704067200,
  "exp": 1704153600
}
```

Biztonsági intézkedések:

- Token csak HTTPS-en keresztül küldhető
 - Token soha nem kerül URL-be vagy cookie-ba (localStorage-ban tárolt, XSS elleni védelem szükséges)
 - Bejelentkezéskor token automatikusan frissül
-

2. Jelszavak Titkosítása

bcrypt

- **Hol:** `Backend/service/helpers/security.helper.js`
- **Salt Rounds:** 10
- **Folyamat:**

```
jelszó (cleartext)
→ salt generálás (bcrypt.genSalt(10))
→ hash (bcrypt.hash(jelszó, salt))
→ adatbázisban tárolás (login_password_hash)
```

Verifikáció:

```
const isMatch = await bcrypt.compare(providedPassword, storedHash)
```

Biztonsági okok:

- Rainbow table támadás ellen (salt)
- Brute force ellen (slow hashing)
- Jelszó soha nem kerül szöveggént az adatbázisba
- Jelszó soha nem logolódik

3. Jelszó-Visszaállítási Tokenek

Secure Token + bcrypt

- **Token Generálás:** `crypto.randomBytes(32).toString('hex')` = 64 karakter hex
- **Tárolás:** bcrypt hash (10 salt rounds)
- **Lejárat:** 24 óra
- **Egy felhasználásos:** Token nullázódik felhasználás után

Folyamat:

1. Felhasználó "Jelszó elfelejtés" kattint
2. Backend generál random token-t
3. Token hashelve tárolódik az adatbázisban
4. Email küldés reset link-kel: `/reset-password?token=XYZ`
5. Felhasználó kattint a linkre
6. Frontend POST `/api/user/reset-password` + token + új jelszó
7. Backend verifikálja a token (bcrypt.compare)
8. Lejárat ellenőrzése
9. Új jelszó bcrypt-tal hashelve
10. Token nullázódik (`password_recovery_hash = NULL`)

4. API Hibakezelés és Error Codes

Hibakódok (Backend → Frontend)

Minden hiba `errorCode` mezőben kerül vissza, a frontend `translateError()` funkcióval fordítja le.

Hibakódok lista (validation-messages.js):

```

{
  // User hibák
  USERNAME_TAKEN: 'Ez a felhasználónév már foglalt',
  EMAIL_TAKEN: 'Ezzel az email címmel már regisztráltak',
  INVALID_CREDENTIALS: 'Hibás felhasználónév vagy jelszó',
  USER_NOT_FOUND: 'Felhasználó nem található',

  // Token hibák
  INVALID_TOKEN: 'Érvénytelen vagy lejárt link',
  TOKEN_EXPIRED: 'A link lejárt, kérj újat',

  // Password hibák
  INVALID_CURRENT_PASSWORD: 'A jelenlegi jelszó helytelen',
  PASSWORD_CHANGE_FAILED: 'Nem sikerült megváltoztatni a jelszót',

  // Auth hibák
  AUTH_REQUIRED: 'Bejelentkezés szükséges',

  // Egyéb
  SERVER_ERROR: 'Szerverhiba történt, próbáld újra később'
}

```

Felhasználó nem látja:

- Valós adatbázis hibákat (SQL, kapcsolat hiba)
- Stack trace-t
- Belső szerver logot

5. CORS (Cross-Origin Resource Sharing)

Backend beállítás (app.js):

```

app.use(cors({
  origin: process.env.FRONTEND_URL || 'http://localhost:5173',
  credentials: true
}))

```

Védelem:

- Csak az engedélyezett frontend domain-ek férhetnek hozzá az API-hoz
- Credentials (cookies) csak az engedélyezett origen-ről

6. Input Validáció

Backend Validáció

Routes → validateRequired() → validateEmailTemplate() → Controller

Validáció típusok:

- Kötelezőség ellenőrzése
- Email formátum validáció
- Jelszó hossz validáció
- Type checking

Frontend Validáció

- Email regex: `/^[^\s@]+@[^\s@]+\.[^\s@]+$/`
- Jelszó: min 8 char, nagybetű, kisbetű, szám
- Felhasználónév: 3-20 char, alfanumerikus + underscore

7. Adatbázis Biztonság

SQL Injection Védelem

- **ORM:** Sequelize (prepared statements)

Jellemző query:

```
// BIZTONSÁGOS - Sequelize használat
const user = await User.findOne({ where: { account_id: userId } })

// VESZÉLYES (nem használt) - Raw SQL
const user = await sequelize.query(`SELECT * FROM account WHERE account_id = ${userId}`)
```

Jelszó Tárolás

- Jelszavak soha nem logolódnak
- Jelszavak soha nem jelennek meg api válaszbán
- Jelszavak soha nem tárolódnak szövegben

8. E-mail Biztonság

Email Token Lehetőség

- Email verifikáció középosztály támogatott (emailTemplate middleware)
- Email küldés Nodemailer-rel
- Reset linkekben token included

9. File Upload Biztonság

Kép Feltöltés Validáció

```
// Frontend validáció
- Max fájl méret: 10MB
- Engedélyezett formátumok: JPG, PNG
- Fájl név felülírás előfűzéssel

// Backend kezelés
- Ideiglenes fájl (/tmp)
- Feldolgozás után azonnal törlés
- Soha nem kerül tárolt tárolóhelyen
```

10. Logging és Monitoring

Loggolt Események

```
Sikeres bejelentkezés
Sikertelen bejelentkezés (username nem létezik, jelszó hibás)
Sikeres regisztráció
Jelszó változtatás
AI elemzés kezdete és vége
Adatbázis hibák
Jelszavak soha nem logolódnak
JWT tokenek soha nem logolódnak
```

Szerver log:

```
Created new skin analysis for user 1: { skin_type: 'oily', skin_problems: [...] }
Updated existing skin analysis for user 1: ...
Failed to save skin analysis to database: ...
```

Titkosítás és Hitelesítés

Titkosított Adatok

Adat	Módszer	Hely	Lejárat
Jelszó	bcrypt (10 salt rounds)	account.login_password_hash	-
JWT Token	HMAC SHA256	frontend localStorage	24 óra

Adat	Módszer	Hely	Lejárat
Jelszó Reset Token	bcrypt (10 salt rounds)	account.password_recovery_hash	24 óra
Kép adatok	Nincs (azonnal törlődik)	/tmp	-
Bőr analízis JSON	Nincs	skin_analysis.skin_problems	-

Nem Titkosított Adatok

Adat	Oka	Miért OK
Felhasználónév	Azonosítás céljából	Nyilvános, csak login_name-hez köthető
Email	Kommunikáció céljából	Felhasználó saját email-je
Bőrtípus	Kijelzés céljából	Nem személyazonosító
Szín-Évszak	Ajánlások céljából	Nem érzékeny adat

Machine Learning Modellek

Modellek Listája

1. Bőrtípus Predikció Model

- **Fájl:** ML/models/skin_accurate_model.keras
- **Kimenet:** 4 osztály (oily, dry, normal, combination)
- **Framework:** TensorFlow/Keras
- **Accuracy:** ~92-98%

Felhasználás:

```
from tensorflow import keras
model = keras.models.load_model('skin_accurate_model.keras')
predictions = model.predict(preprocessed_image)
skin_type = ['oily', 'dry', 'normal', 'combination'][np.argmax(predictions)]
```

2. Bőrprobléma Detektálás Model

- **Fájl:** ML/models/skin_problems_model.keras
- **Kimenet:** Multi-label klasszifikáció (5 label)
- **Címkék:** Acne, Bags, Milia, Redness, WhiteHead
- **Framework:** TensorFlow/Keras
- **Accuracy:** ~90-95%

Felhasználás:

```
model = keras.models.load_model('skin_problems_model.keras')
predictions = model.predict(preprocessed_image)
problems = [label for label, pred in zip(labels, predictions[0]) if pred > 0.5]
```

Python Scripts

1. predict_app.py

- Bőrtípus predikció
- Szeparáció az előfeldolgozásra
- JSON output

2. predict_skin_problems.py

- Bőrprobléma detektálás
- Multi-label output
- Confidence scores

3. train.py

- Model betanítás
- Data augmentation
- Validation split: 80/20

4. train_skin_problems.py

- Bőrprobléma model tanítása
- Imbalanced data kezelés

Python-Node.js Integráció

Folyamat:

```
Frontend (image)
  ↓ (multipart/form-data)
Backend (Express)
  ↓ (child_process.spawn)
Python Script
  ↓ (exec)
ML Model
  ↓ (predictions)
Python Script
  ↓ (JSON output)
Backend
  ↓ (parse results)
```


Database
↓ (store)
Frontend
↓ (display)
User

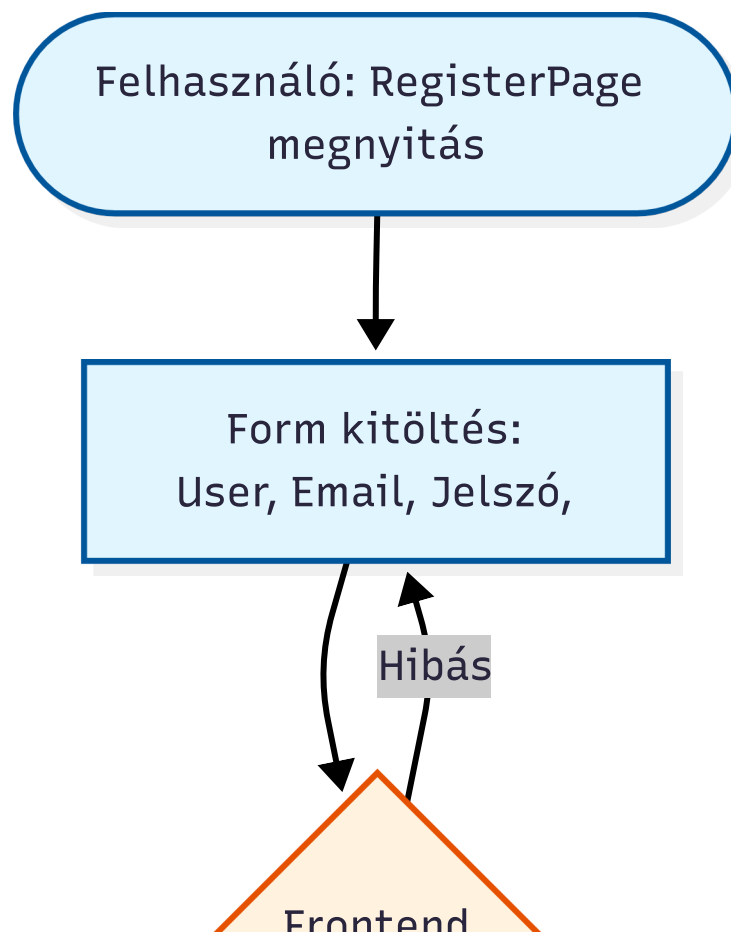
Kód (Backend):

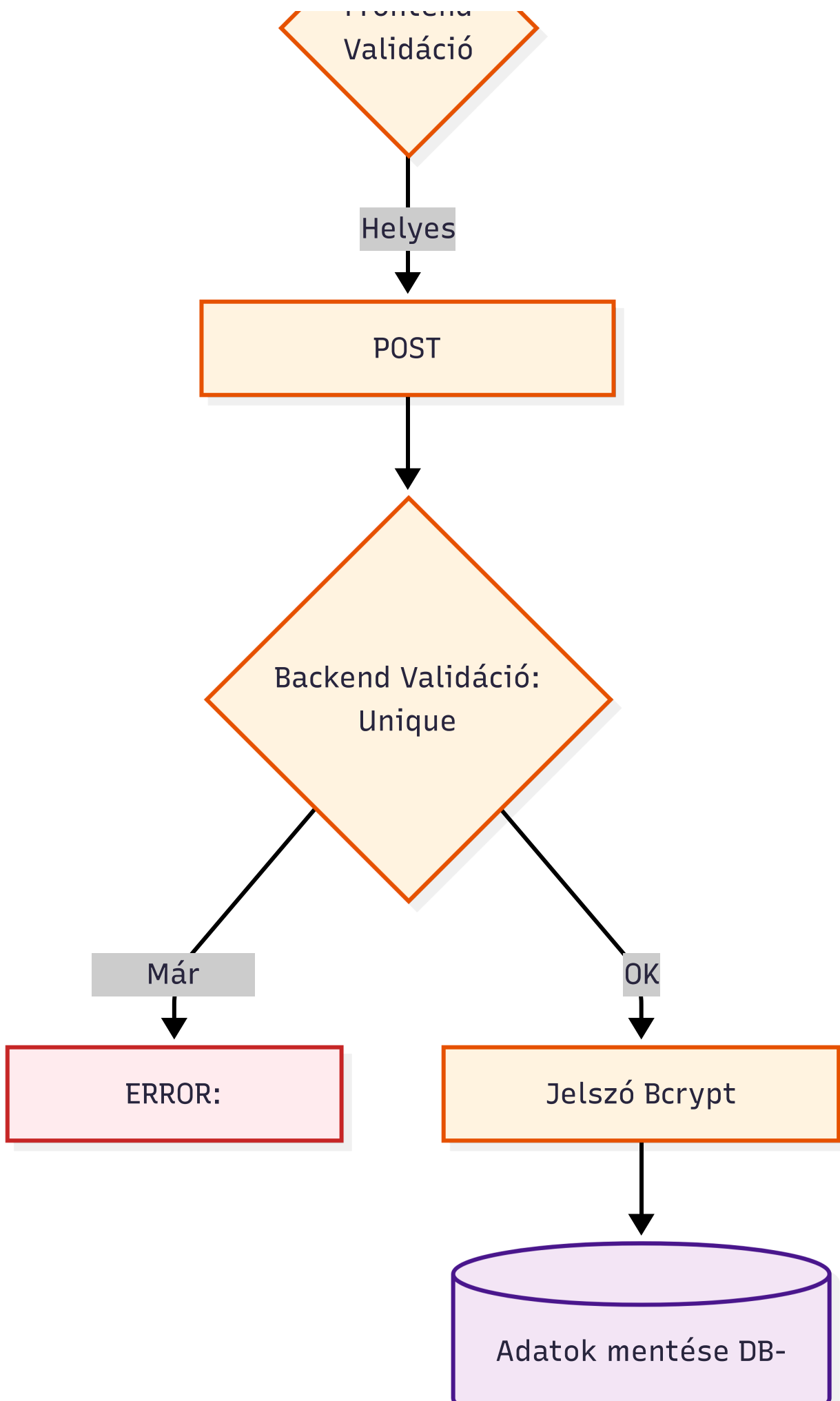
```
const { exec } = require('child_process');

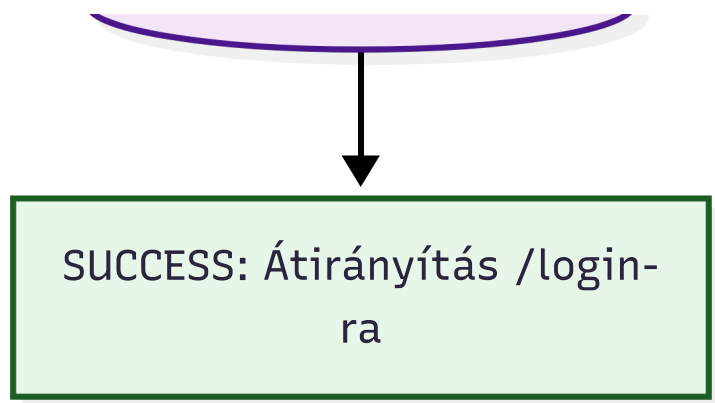
// Python script meghívása
exec(`python Backend/models/predict_app.py "${imagePath}"`, (error, stdout,
stderr) => {
  if (error) {
    // Error handling
  }
  const predictions = JSON.parse(stdout);
  // Felhasználás...
});
```

Felhasználói Folyamatok

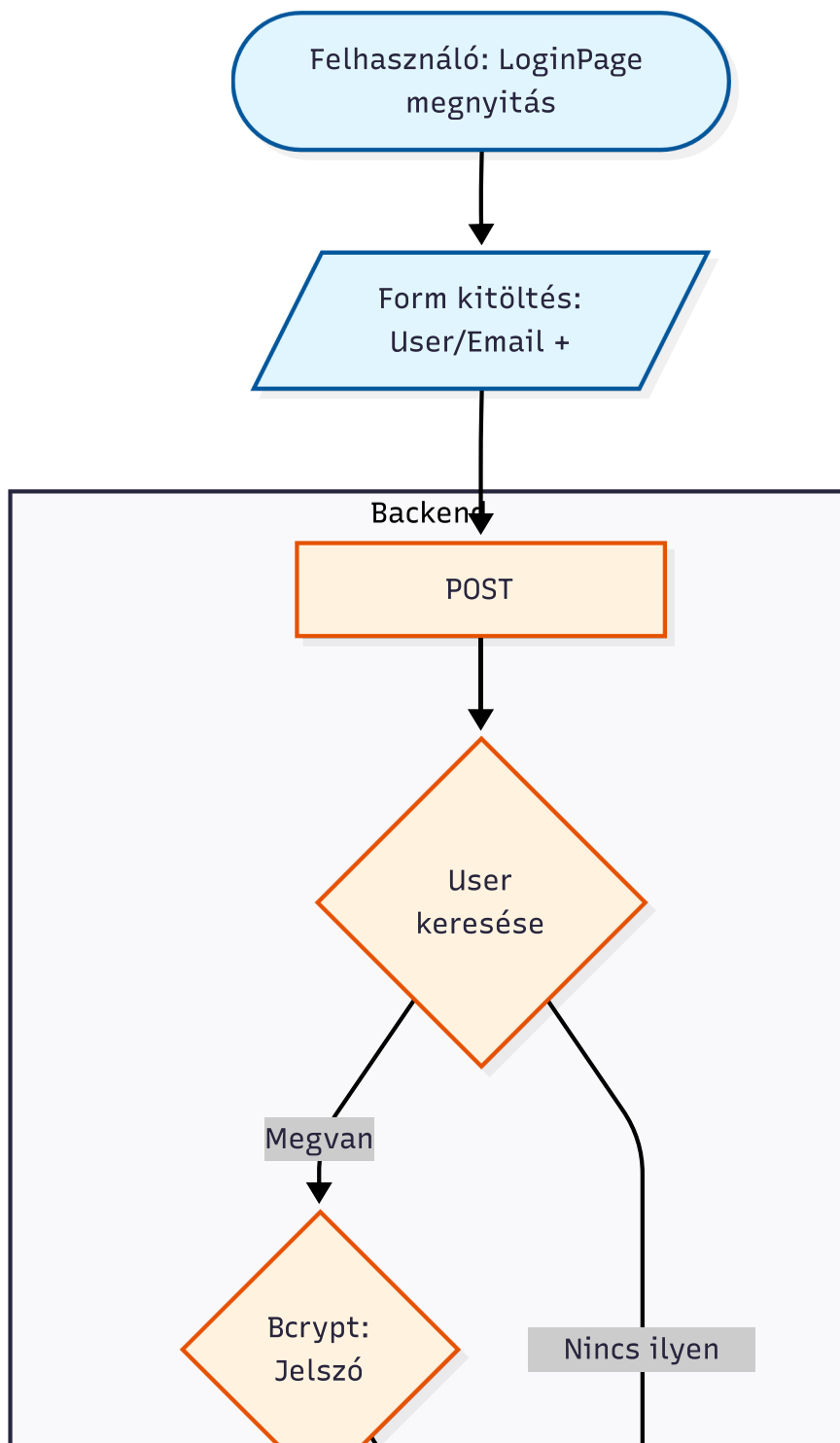
1. Regisztráció Flow

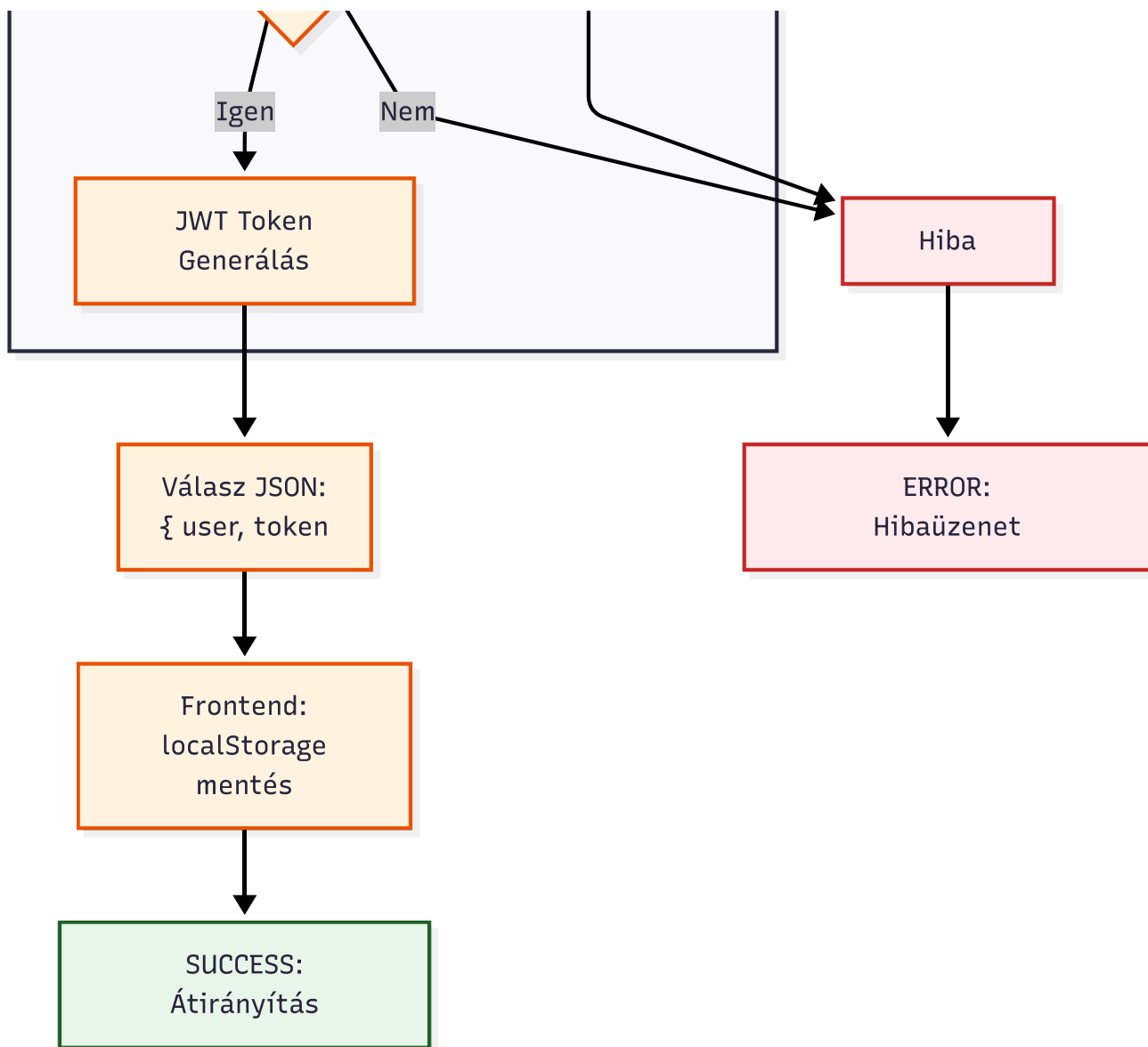




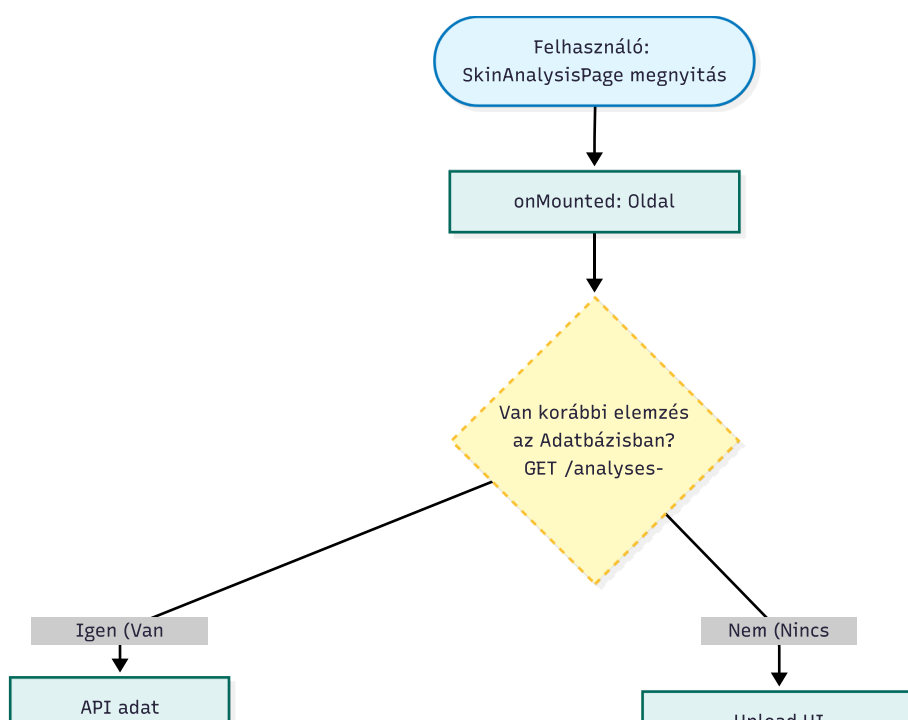


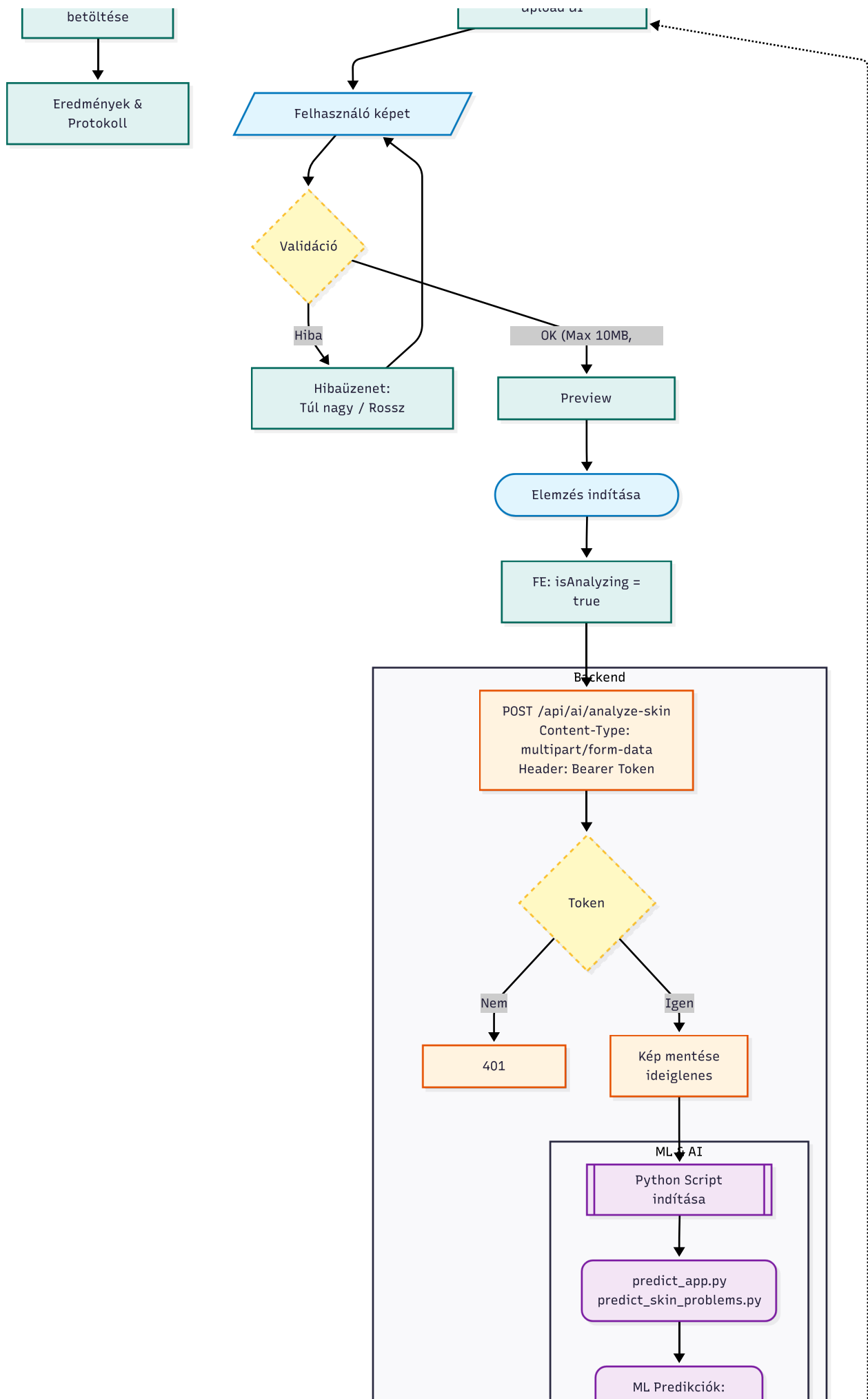
2. Bejelentkezés Flow

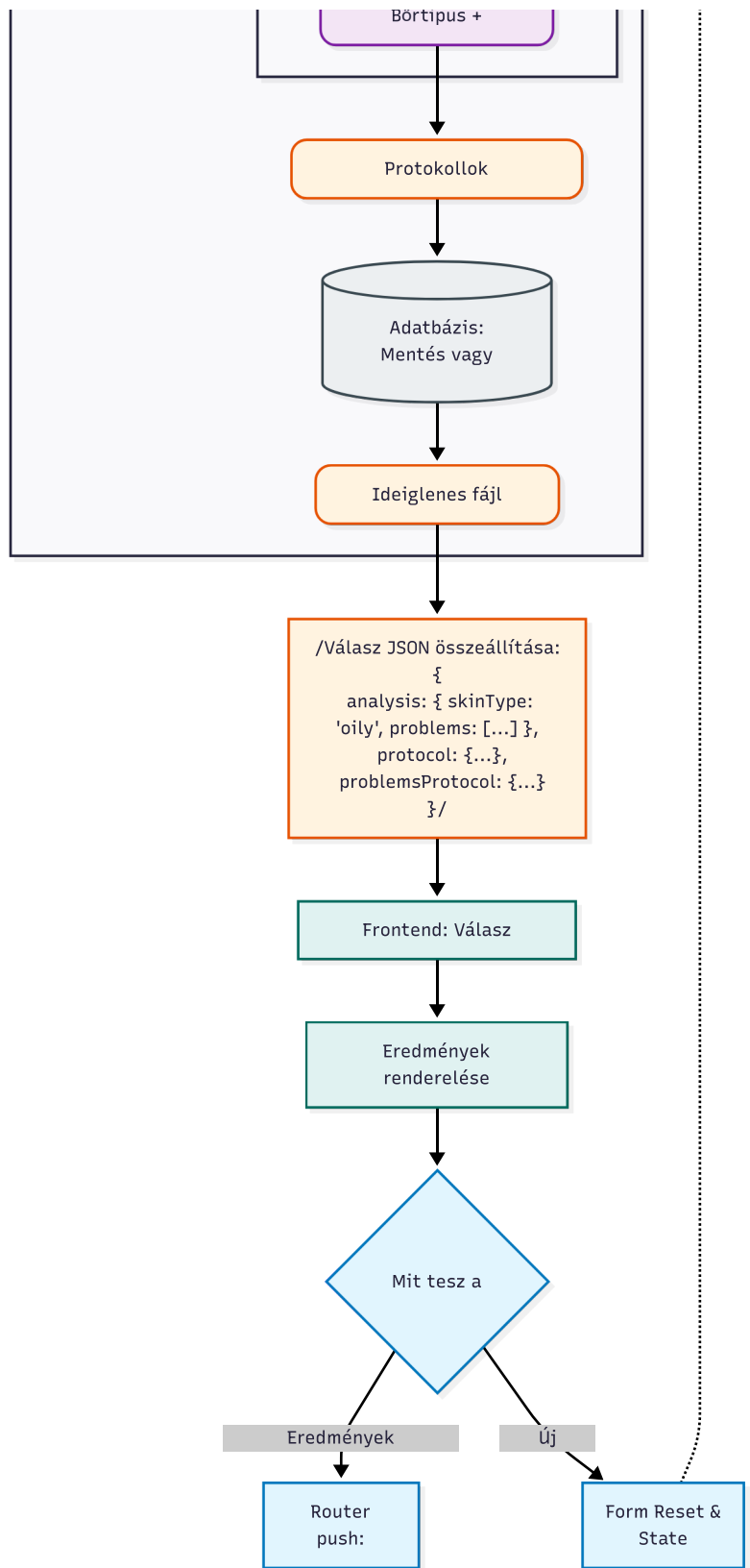




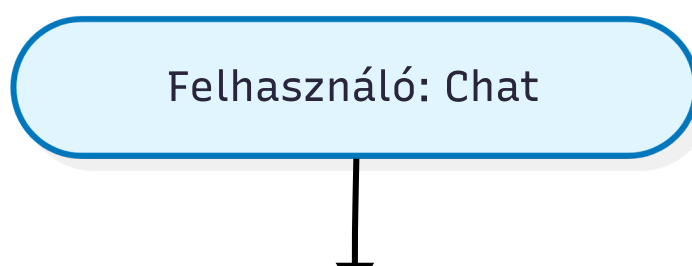
3. Bőrelemzés Flow

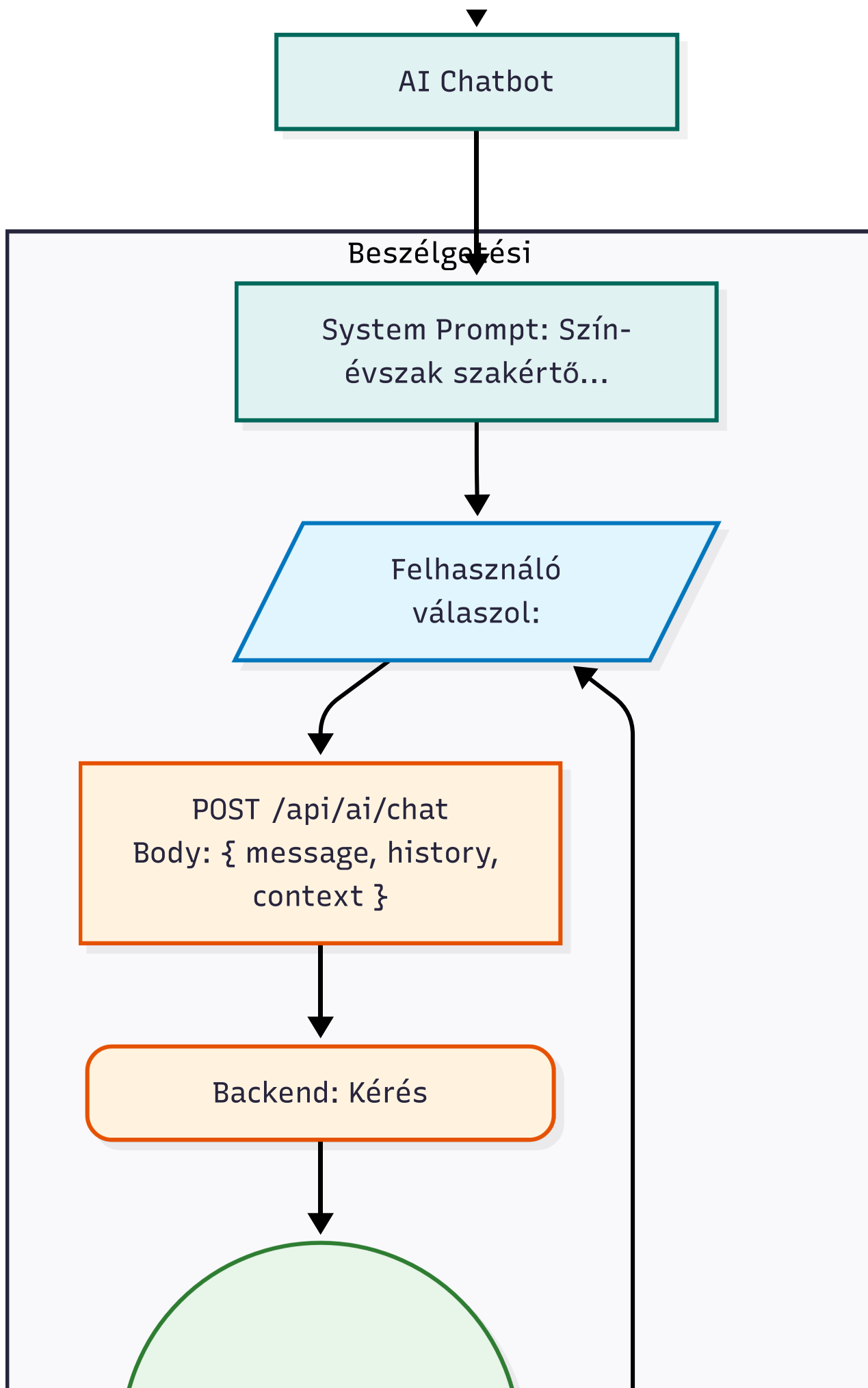


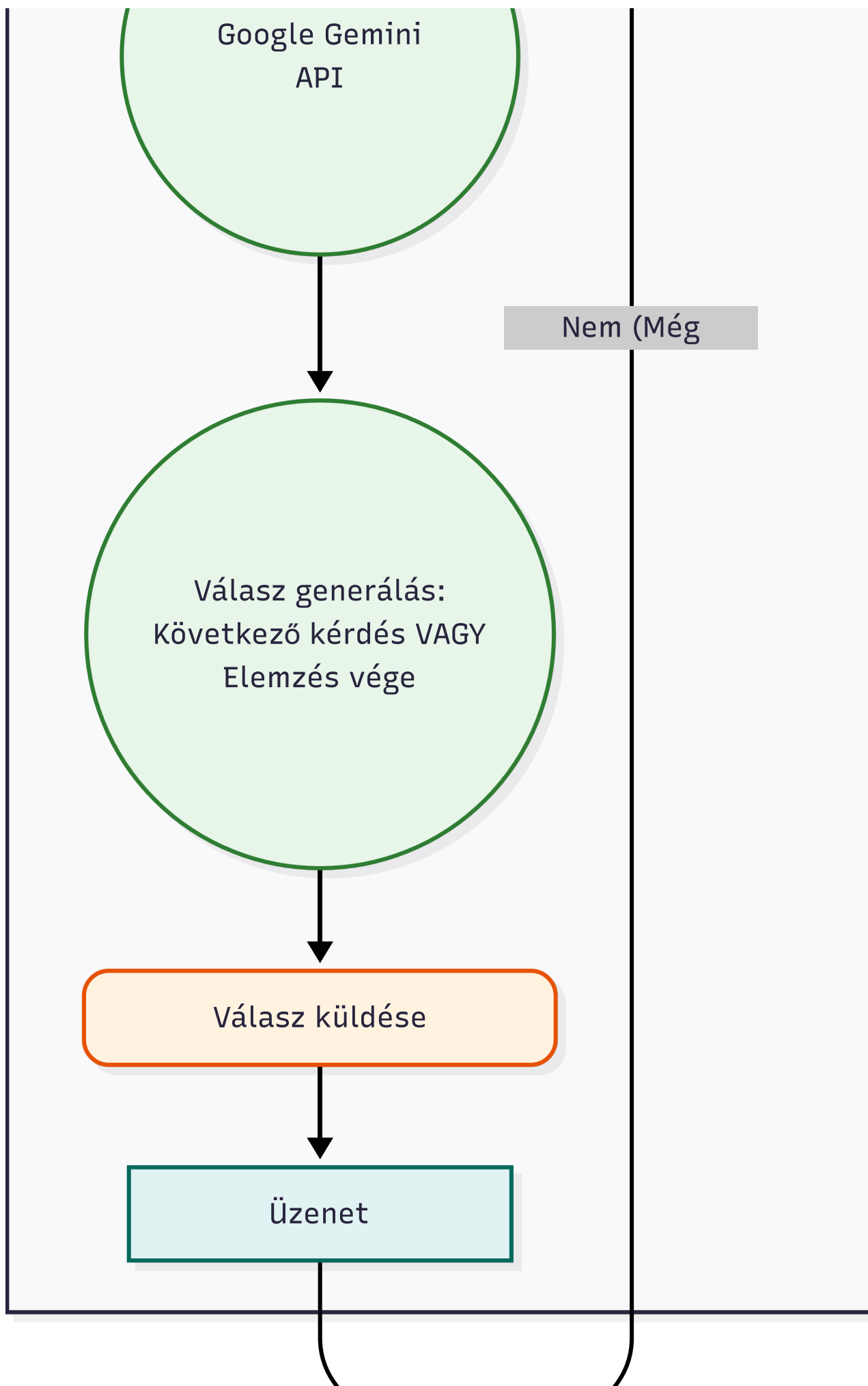


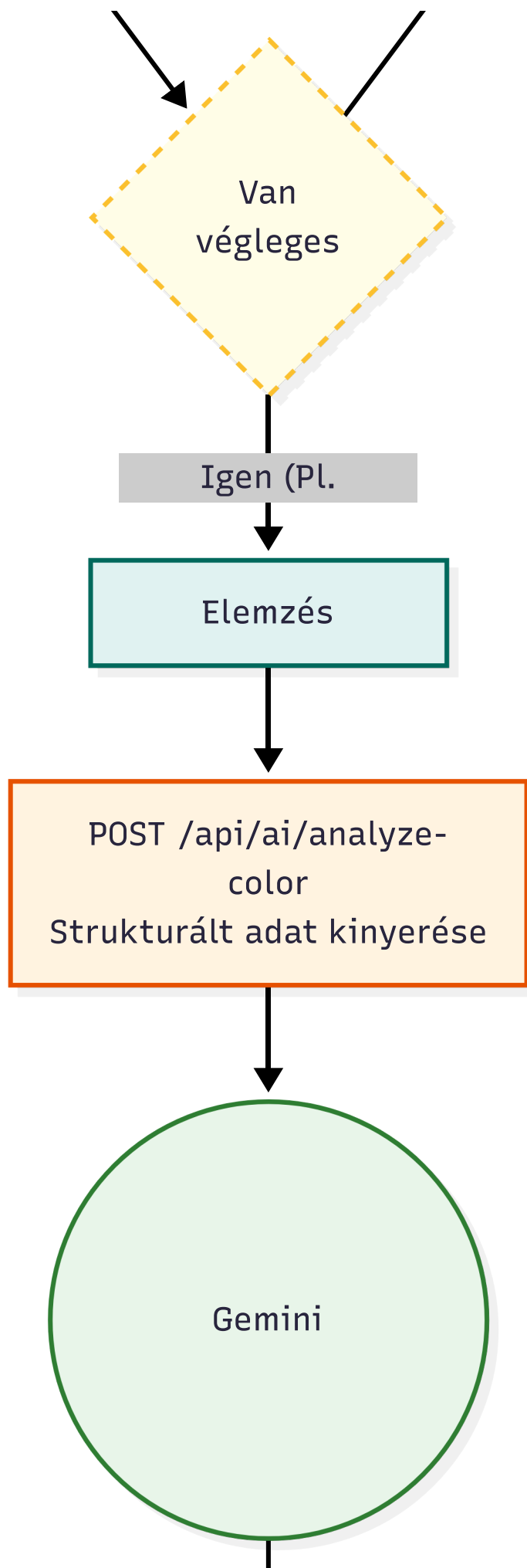


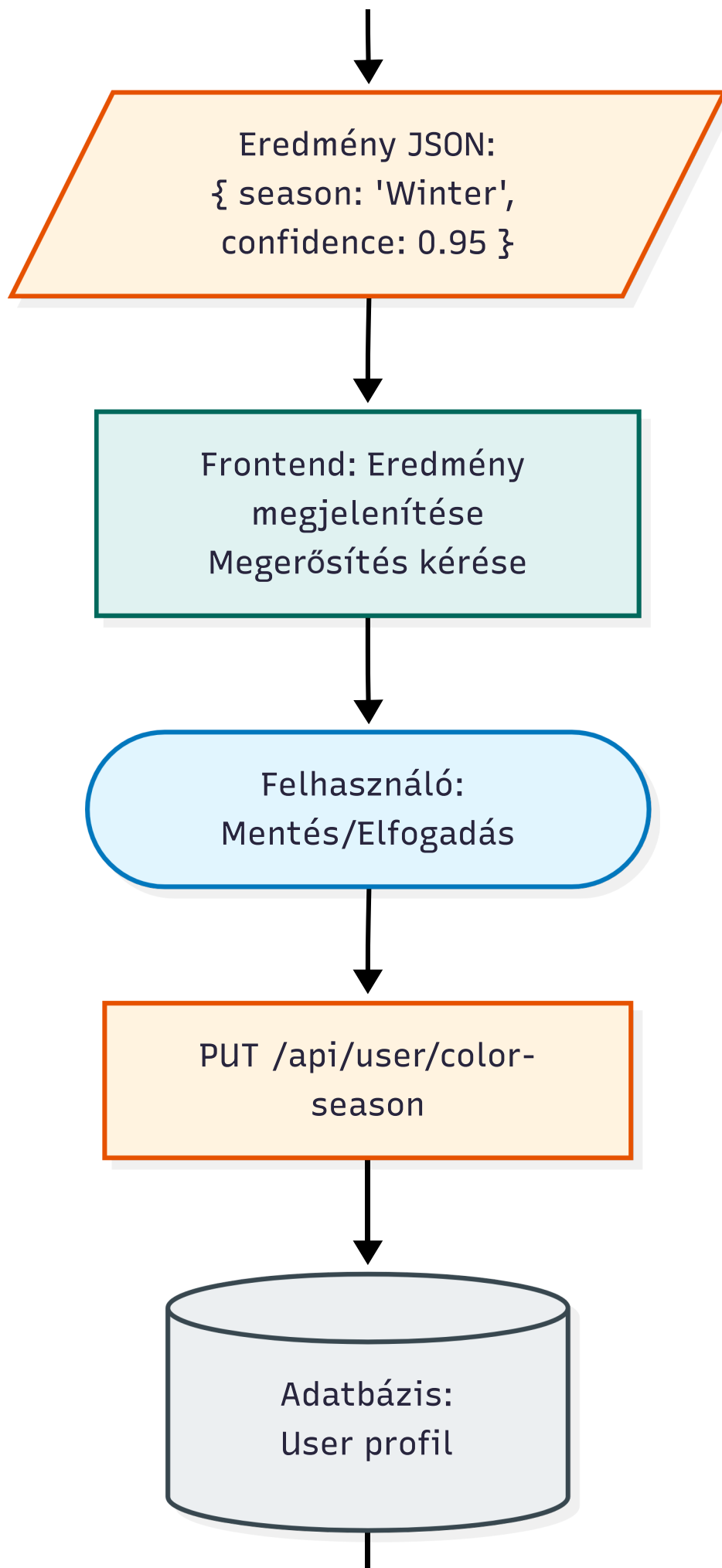
4. Szín-Évszak Meghatározás Flow

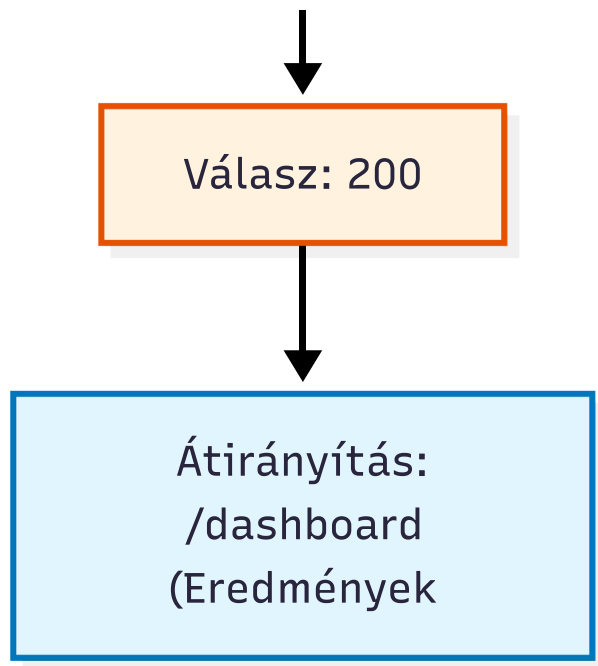




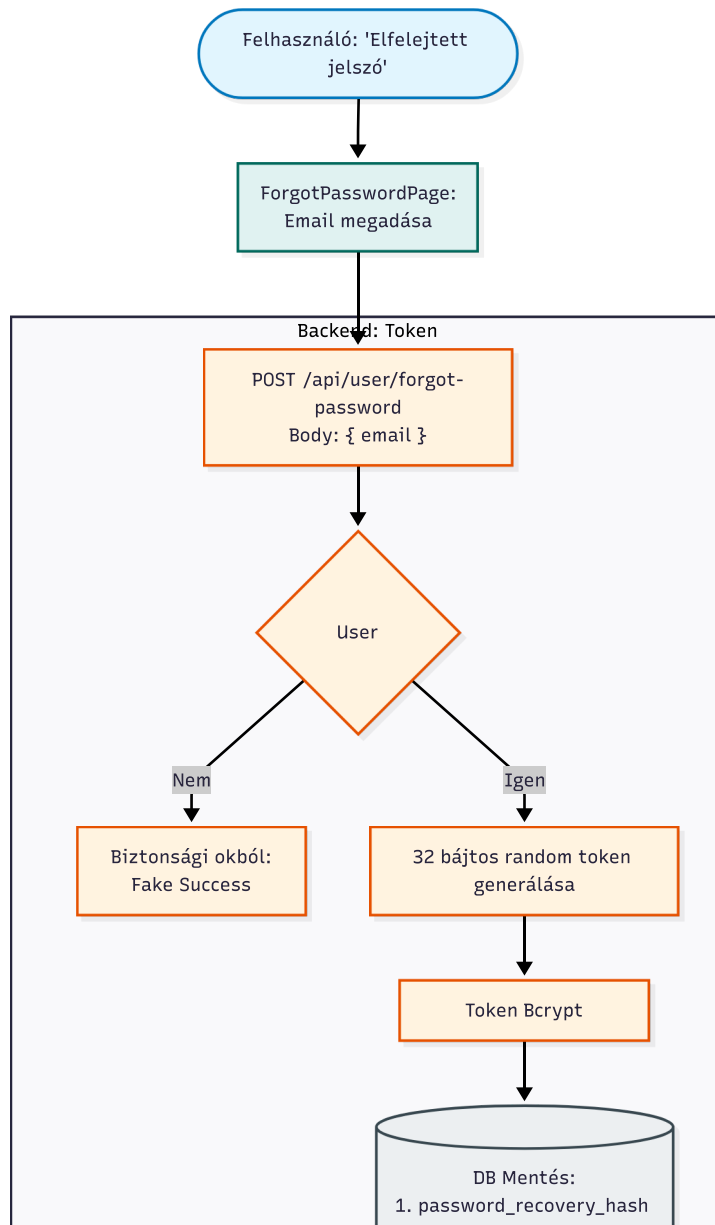


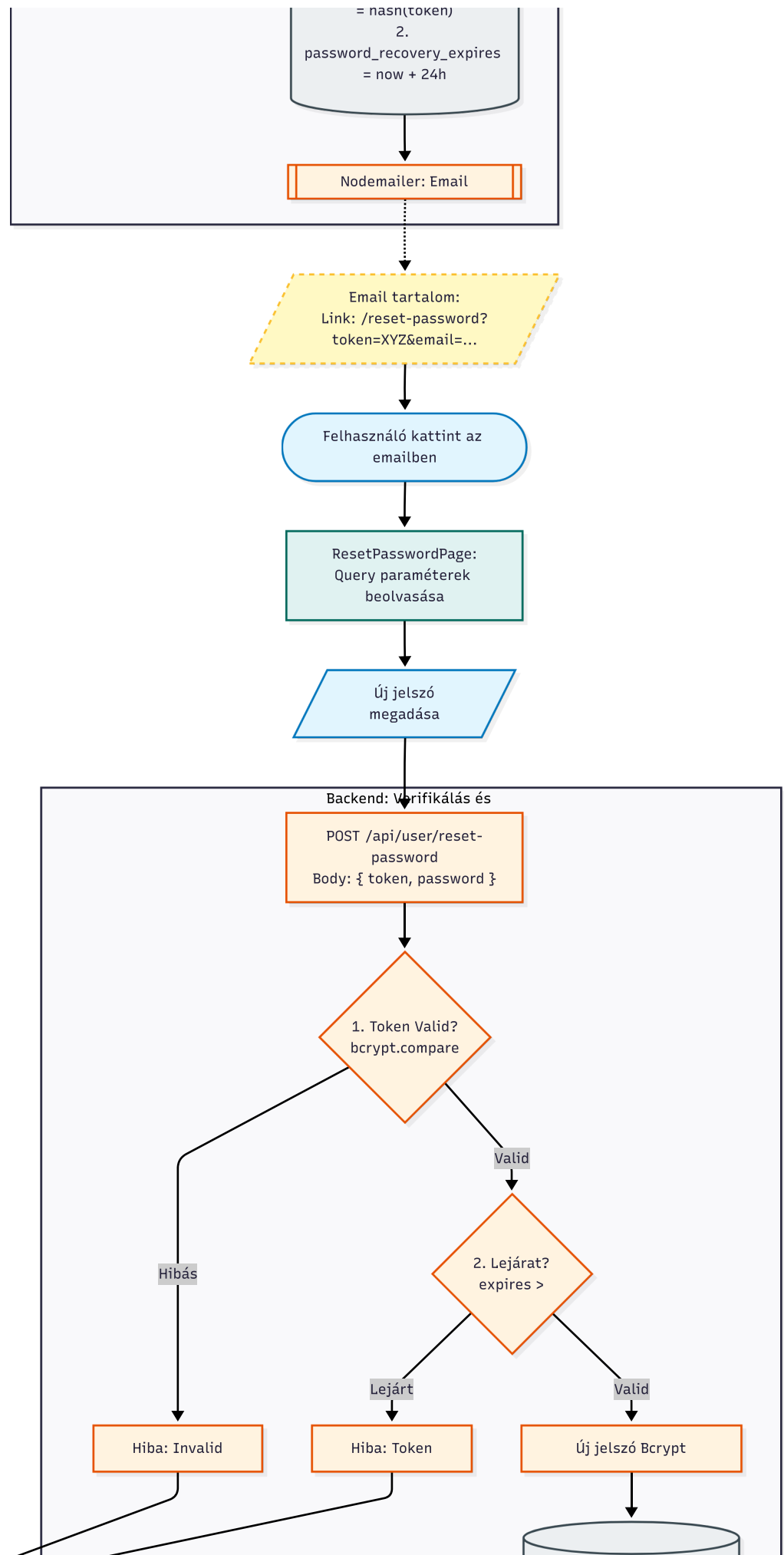






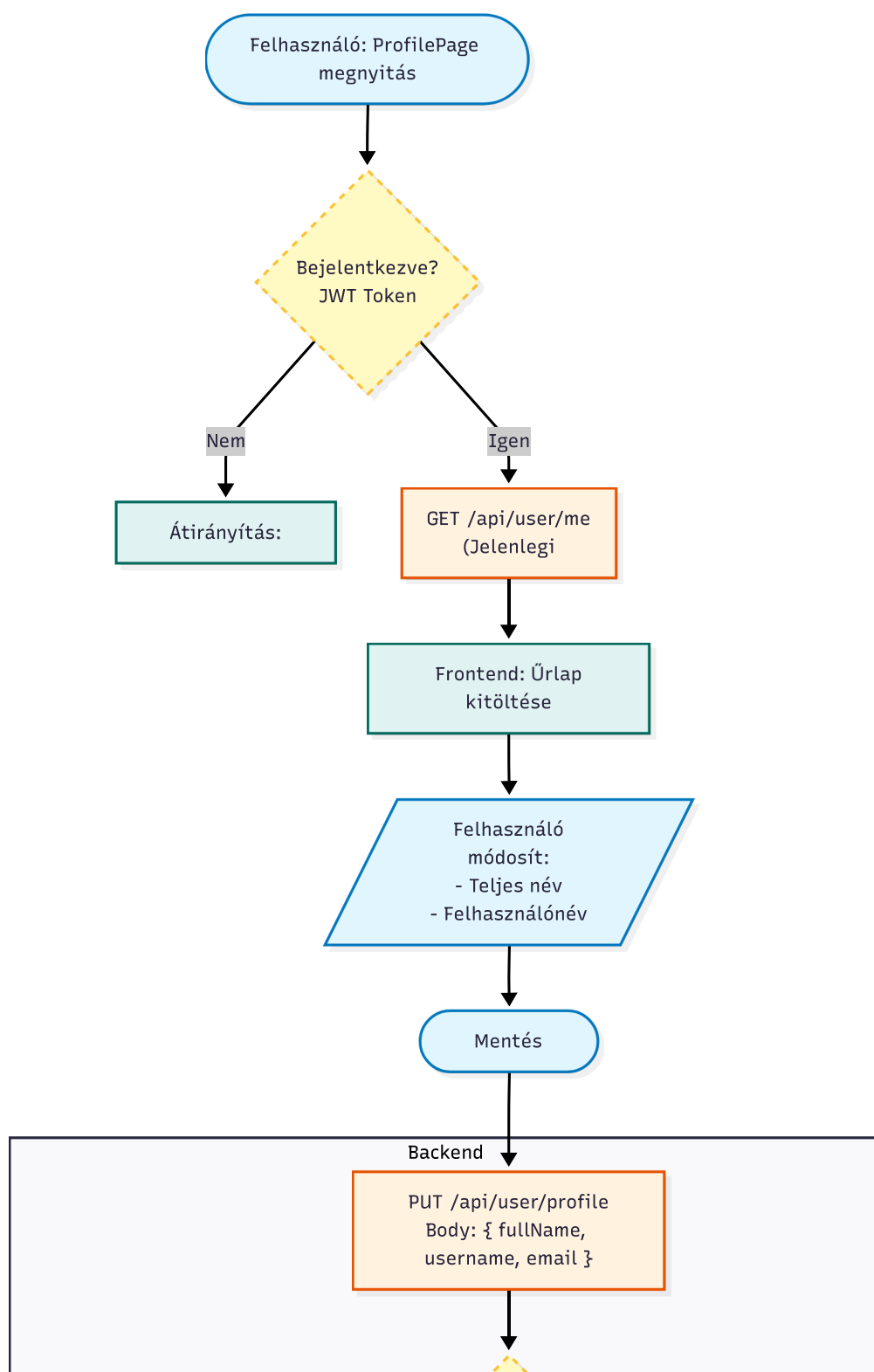
5. Jelszó-Elfelejtés Flow

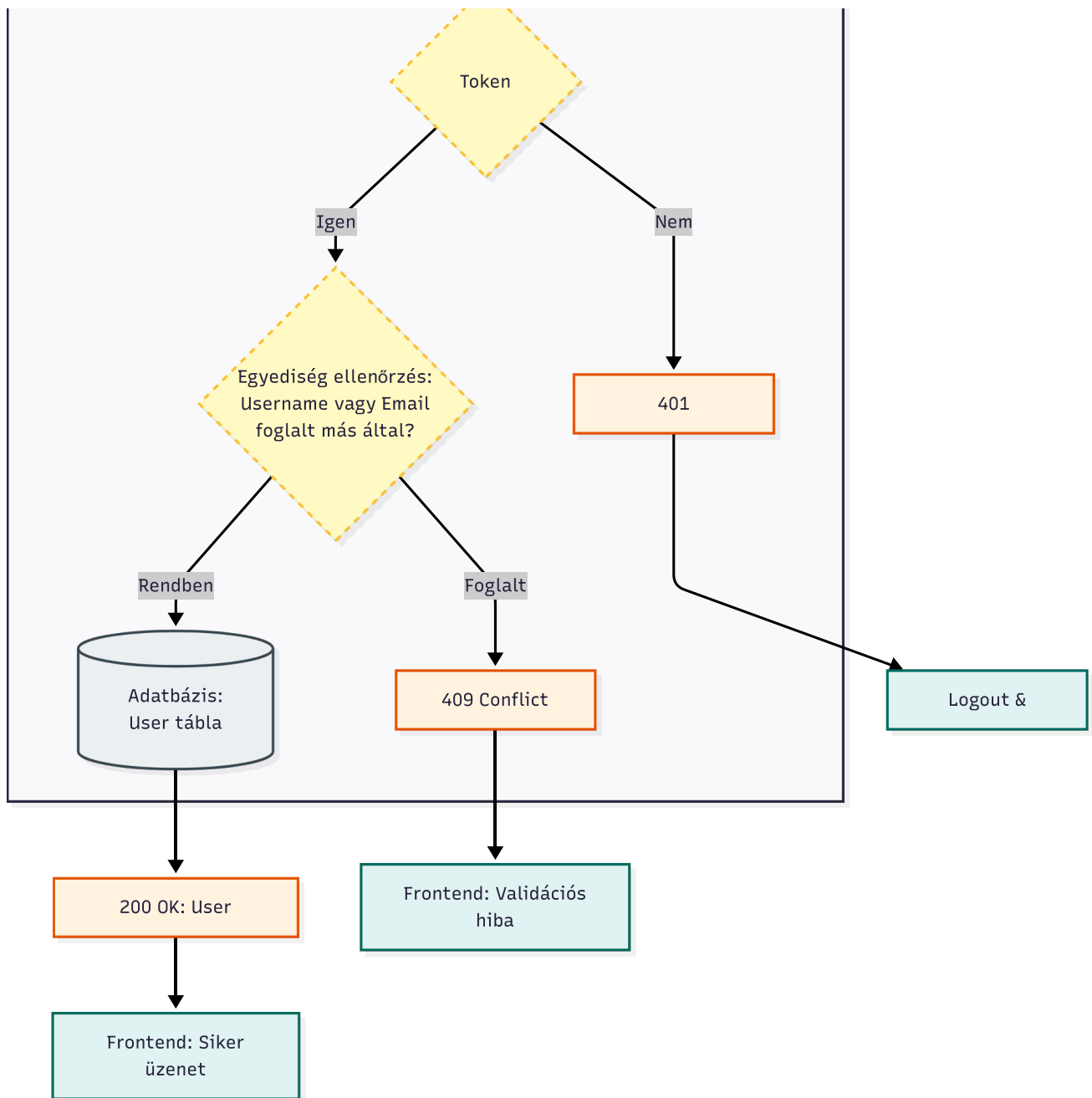






6. Profil Módosítás Flow





Telepítés és Futtatás

Előfeltételek

- Node.js
- Python
- MySQL
- npm vagy yarn

Backend Telepítés

```
cd Backend
```

```
# Függőségek
```

```
npm install

# Environment változók (.env file)
JWT_SECRET=your_secret_key
DATABASE_URL=mysql://user:pass@localhost:3306/webfej1
FRONTEND_URL=http://localhost:5173
GOOGLE_GEMINI_API_KEY=your_api_key

# Szerver indítása
npm start
# vagy dev módban
npm run dev # nodemon automatikus restart
```

Frontend Telepítés

```
cd Frontend

# Függőségek
npm install

# Development server
npm run dev
# http://localhost:5173 megnyitódik

# Production build
npm run build
```

Python ML Setup

```
cd ML

# Virtual environment
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate    # Windows

# Függőségek
pip install tensorflow numpy pandas

# Tesztelés
python ML/scripts/predict_app.py <image_path>
```